

AI Knowledge Assistant — Project Proposal

A realistic project proposal designed as a business-style document for multi-document RAG learning.

1. Executive Summary

This proposal describes the development of an AI Knowledge Assistant that allows users to ask natural-language questions about an organization's internal technical documents. The system will use Retrieval-Augmented Generation (RAG) to retrieve relevant document passages and generate grounded answers.

Business problem

Organizations often store information in PDFs, reports, manuals, proposals, and technical documents. Employees may know that information exists but spend significant time searching for the right page or section. Keyword search can also fail when a question uses different wording from the source document.

Proposed solution

Build a document question-answering application that ingests approved PDF files, creates searchable vector representations, retrieves relevant passages, and uses a language model to produce answers with source references.

Expected outcome

Users should be able to ask questions in natural language and receive concise answers grounded in the available document collection.

2. Objectives and Scope

Project objectives

- Create a reliable PDF ingestion pipeline.
- Extract and clean text while preserving document metadata.
- Split documents into retrieval-friendly chunks.
- Generate embeddings and store them in a vector database.
- Retrieve relevant evidence for user questions.
- Generate answers using an LLM with explicit grounding instructions.
- Display document and page information for retrieved evidence.
- Evaluate retrieval and answer quality using a defined test set.

In scope

- PDF document upload and ingestion.
- Text extraction and chunking.
- Embedding generation.
- Vector search.
- Question answering.
- Source tracking.
- Basic user interface.
- Evaluation and error analysis.

Out of scope for the first version

- Automatic editing of source documents.
- Unrestricted access to confidential data.
- Autonomous decision-making.
- Guaranteeing correctness when source documents are incomplete or contradictory.

3. Proposed System Architecture

High-level architecture

- Document layer → PDF files are uploaded and validated.
- Parsing layer → text, page numbers, and metadata are extracted.
- Chunking layer → documents are divided into meaningful passages.
- Embedding layer → chunks are converted into vectors.
- Vector store → vectors, text, and metadata are indexed.
- Retrieval layer → the user's query is embedded and similar chunks are retrieved.
- Generation layer → retrieved evidence is inserted into an LLM prompt.
- Application layer → the answer and source information are displayed.

Data flow

PDF → extracted text → chunks → embeddings → vector database → user query → query embedding → retrieval → context assembly → LLM → grounded response

Recommended metadata

Metadata	Purpose
document_id	Unique document identification.
file_name	Original PDF name.
page_number	Allows users to locate source material.
chunk_id	Unique chunk identification.
section	Supports structure-aware retrieval.
created_at	Tracks ingestion information.

4. Functional Requirements

ID	Requirement
FR-01	System shall accept supported PDF files.
FR-02	System shall extract text and retain page-level metadata where possible.
FR-03	System shall split documents into configurable chunks.
FR-04	System shall generate embeddings for document chunks.
FR-05	System shall store and search embeddings.
FR-06	System shall retrieve top-k relevant chunks for a query.
FR-07	System shall generate answers using retrieved context.
FR-08	System shall show source document and page information.
FR-09	System shall provide an appropriate response when evidence is insufficient.

Non-functional requirements

- Reasonable query latency for the target workload.
- Reproducible ingestion and retrieval configuration.
- Secure handling of uploaded documents.
- Clear error messages for unsupported or corrupted files.
- Traceability from generated answer back to retrieved chunks.

5. Evaluation Strategy

A RAG system should not be judged only by whether the final answer sounds convincing. Evaluation should examine retrieval and generation separately.

Retrieval evaluation

- Create a set of representative user questions.
- Identify the expected source document and relevant passage.
- Measure whether the relevant chunk appears in the top-k results.
- Inspect false positives and false negatives.

Generation evaluation

- Correctness: Does the answer agree with the retrieved evidence?
- Faithfulness: Does the answer avoid unsupported claims?
- Relevance: Does it directly address the question?
- Completeness: Does it include important available evidence?
- Abstention: Does it avoid inventing an answer when evidence is missing?

Example test case

Question	Expected source	Expected behavior
What is IoU?	object_detection.pdf	Explain intersection over union and its purpose.
What is cosine similarity?	embeddings.pdf	Explain the formula and intuition.
What is multi-head attention?	attention.pdf	Explain parallel attention heads.
What are the project objectives?	proposal.pdf	Retrieve the objectives section.

6. Risks, Timeline, and Deliverables

Key risks

- Poor PDF extraction can damage retrieval quality.
- Bad chunk boundaries can separate definitions from explanations.
- Weak embeddings can reduce semantic retrieval quality.
- Large prompts can increase latency and cost.
- The LLM may generate unsupported statements if grounding instructions are weak.
- Conflicting documents can produce ambiguous answers.

Suggested 4-week timeline

Week	Activities
Week 1	PDF ingestion, extraction, cleaning, chunking experiments.
Week 2	Embeddings, vector database, retrieval pipeline.
Week 3	LLM integration, prompting, source display, UI.
Week 4	Evaluation, error analysis, optimization, documentation.

Deliverables

- Source code repository.
- Ingestion pipeline.
- Vector index/database.
- RAG application.
- Evaluation dataset and results.
- Technical documentation.
- Demonstration of representative questions.

7. Acceptance Criteria and Future Improvements

Acceptance criteria

- All four project PDFs can be ingested successfully.
- The system can retrieve relevant chunks for representative questions.
- Answers are grounded in retrieved content.
- Source document and page information can be traced for answers.
- The system avoids confidently answering questions when evidence is unavailable.
- The retrieval and generation pipeline is documented and reproducible.

Future improvements

- Hybrid search combining keyword and vector retrieval.
- Reranking retrieved passages.
- Query rewriting for ambiguous questions.
- Conversation-aware retrieval.
- Multimodal PDF processing for tables and figures.
- Access control and document-level permissions.
- Automated RAG evaluation and monitoring.

Conclusion

The proposed AI Knowledge Assistant provides a practical foundation for learning and demonstrating RAG. The project separates document processing, retrieval, and generation so each component can be tested independently. This makes the system suitable both as a learning project and as a portfolio demonstration.

End of proposal